# Q-0001: API de reservas de salas de reunião

Nível 1 (Fundamento) | dotnet | 6 horas estimadas | criada em 2026-09-28

Conceitos: minimal-api, ef-core, validacao, paginacao, migrations

## Contexto

Um coworking com 12 salas de reunião controla as reservas numa planilha compartilhada, e duas equipes já apareceram na mesma sala no mesmo horário. O gerente quer uma API simples que a recepção e um futuro app possam consumir. Cada sala tem nome, capacidade e andar; cada reserva tem sala, responsável, início, fim e número de participantes.

## Requisitos funcionais

1. Cadastrar, listar, consultar por id e desativar salas (desativar não apaga: salas desativadas não aceitam novas reservas).
2. Criar, consultar, listar e cancelar reservas.
3. Não permitir duas reservas da mesma sala com horários sobrepostos (reservas canceladas não contam).
4. Não permitir reserva com número de participantes maior que a capacidade da sala.
5. Reservas devem ter duração entre 15 minutos e 4 horas e começar no futuro.
6. Listagem de reservas com filtros opcionais por sala e por intervalo de datas, ordenada por início.
7. Listagem de reservas paginada (parâmetros pagina e tamanho, tamanho máximo 50).

## Requisitos não funcionais

- Erros de validação retornam 400 com a lista de campos inválidos; recurso inexistente retorna 404; conflito de horário retorna 409.
- A verificação de conflito ao criar reserva continua respondendo em menos de 50 ms com 100 mil reservas na mesma sala.

## Arquitetura: vertical-slice

### Justificativa

A API tem só dois recursos (salas e reservas), mas os casos de uso têm regras bem diferentes entre si: criar reserva concentra quase toda a lógica (conflito de horário, capacidade, duração, sala ativa), enquanto listar é filtro e paginação e cancelar é só troca de estado. Organizar por caso de uso deixa cada regra num lugar só e casa com Minimal APIs, em que cada endpoint já é uma unidade. Com um único banco (SQLite via EF Core) e sem previsão de outras entradas além da API, não há infraestrutura a isolar, e o DbContext do EF já cumpre o papel de unidade de trabalho.

### Alternativas descartadas

- `camadas-tradicional`: Divide o código por tipo técnico: a funcionalidade de criar reserva ficaria espalhada entre endpoint, ReservaService e ReservaRepository, e o service juntaria criar, listar e cancelar numa classe só. O repositório por cima do EF só repetiria métodos que o DbContext já tem.

- `clean-architecture`: Quatro projetos, interfaces e mapeamentos para duas entidades e sete requisitos. A proteção que ela dá (trocar banco, várias entradas) não é necessária aqui; a cerimônia esconderia as regras de negócio, que são o ponto da questão.
- `cqrs`: Leitura e escrita usam o mesmo modelo e o mesmo banco, sem volume que justifique separar. Separar comandos de consultas com uma biblioteca de mediador só adicionaria indireção.

### Regras

1. Cada caso de uso fica em Features/<Recurso>/<CasoDeUso>/, com o request, o response, a validação e a lógica juntos (ex.: Features/Reservas/CriarReserva/).
2. Cada recurso expõe um método de extensão que mapeia suas rotas (ex.: app.MapReservas()); o Program.cs só registra serviços e chama esses métodos.
3. Regras de negócio (conflito, capacidade, duração, sala ativa) ficam no caso de uso ou em métodos da entidade, nunca no Program.cs ou em middleware.
4. Os casos de uso usam o AppDbContext diretamente; não há repositório genérico nem interfaces de repositório.
5. Uma feature não referencia classes de outra feature. Código usado por duas ou mais (ex.: paginação, entidades, DbContext) vai para Shared/ ou Data/.
6. Respostas usam DTOs próprios do caso de uso (records); entidades do EF nunca saem na resposta.
7. Consultas que dependem de volume (conflito, listagem) filtram no banco; nada de ToListAsync() seguido de filtro em memória.

### Estrutura de pastas

```
src/Reservas.Api/
  Program.cs
  Data/          DbContext e migrations
  Domain/        entidades
  Shared/        código usado por mais de uma feature
  Features/
    Reservas/
      CriarReserva/   exemplo de fatia: request, response, validação e lógica juntos
      ...             as demais fatias de Reservas e de Salas você define a partir dos requisitos
```

### Convenções

- Nullable reference types habilitado e sem avisos de nulidade no build.
- Namespaces por arquivo (file-scoped) e um tipo público por arquivo.
- Nomes de domínio em português, iguais às rotas (Sala, Reserva, CriarReserva); termos técnicos em inglês (Endpoint, Request, Response, DbContext).
- Métodos assíncronos terminam em Async e recebem CancellationToken, repassado ao EF Core.
- DTOs de entrada e saída são records imutáveis.
- Arquivo .editorconfig na raiz; dotnet format --verify-no-changes passa sem alterações.

### Trade-offs

- Aceita alguma duplicação entre fatias (ex.: dois casos de uso buscando a sala do mesmo jeito) em troca de cada funcionalidade ser independente. Extraia para Shared/ só quando a mesma lógica aparecer pela terceira vez.

- Se o sistema crescer para várias entradas (worker de lembretes, app móvel com regras próprias) ou para regras de domínio compartilhadas por muitos casos de uso, vale migrar o núcleo para uma camada de domínio isolada.

## Requisito algorítmico

| | |
|---|---|
| **Ponto** | Verificação de conflito de horário em CriarReserva |
| **Modo** | imposto |
| **Estruturas** | indice-btree-banco, intervalos-sobreposicao |
| **Tempo** | O(log n + k) |
| **Espaço** | O(1) |
| **n e k** | n = reservas já cadastradas da sala (até 100 mil ao longo de 5 anos); k = reservas devolvidas pela faixa consultada (poucas, porque uma reserva dura no máximo 4 horas). |

### Justificativa

A verificação roda a cada nova reserva. Carregar as reservas da sala para a memória, ou deixar o banco varrer a tabela, custa O(n) e fica mais lento a cada mês de uso, justamente na operação que a recepção faz o dia inteiro. Com um índice composto e uma faixa de busca limitada pela duração máxima, o custo não cresce com o histórico.

### Como será verificado

1. Teste que cadastra 100 mil reservas numa sala e mede POST /reservas: responde em menos de 50 ms.
2. EXPLAIN QUERY PLAN da consulta de conflito no SQLite mostra SEARCH ... USING INDEX, e não SCAN.
3. O log de SQL do EF Core mostra uma única consulta filtrada no banco, sem trazer as reservas da sala para a memória.
4. Seção "Análise de complexidade" no README explicando a consulta, o índice e por que o custo é O(log n + k).

## Restrições

### Obrigatório

- Minimal APIs (sem controllers)
- EF Core com SQLite
- Migrations para criar o banco

### Proibido

- Bibliotecas de mapeamento automático (AutoMapper, Mapster)
- Expor as entidades do EF diretamente nas respostas; use DTOs

### Outras

- Sem autenticação nesta questão
- Sem front-end

## Entregáveis

- Projeto em solucoes/Q-0001/
- README com comando para rodar, exemplos de requisição (arquivo .http ou curl) e seção "Análise de complexidade"

## Critérios de aceite

1. POST /reservas com horário que sobrepõe outra reserva ativa da mesma sala retorna 409.
2. POST /reservas com 20 participantes numa sala de capacidade 8 retorna 400 indicando o campo participantes.
3. Cancelar uma reserva libera o horário: uma nova reserva no mesmo intervalo é aceita.
4. GET /reservas?pagina=2&tamanho=10 retorna no máximo 10 itens e informa o total de registros.
5. GET /reservas?salaId=3&de=2026-10-01&ate=2026-10-07 retorna só reservas da sala 3 que começam no intervalo.
6. Sala desativada recusa novas reservas com 400 ou 409 e mensagem clara.
7. O banco é criado do zero rodando as migrations.
8. A estrutura de pastas segue a definida em arquitetura.estrutura_pastas e nenhuma pasta em Features/ importa namespace de outra feature.
9. dotnet build sem avisos de nulidade e dotnet format --verify-no-changes sem alterações.
10. Com 100 mil reservas numa sala, criar uma nova reserva responde em menos de 50 ms, e o plano da consulta de conflito usa índice.

## Desafios extras (opcionais)

- Endpoint que sugere o próximo horário livre de uma sala para uma duração pedida.
- Impedir condição de corrida: duas requisições simultâneas para o mesmo horário não podem ambas ser aceitas.